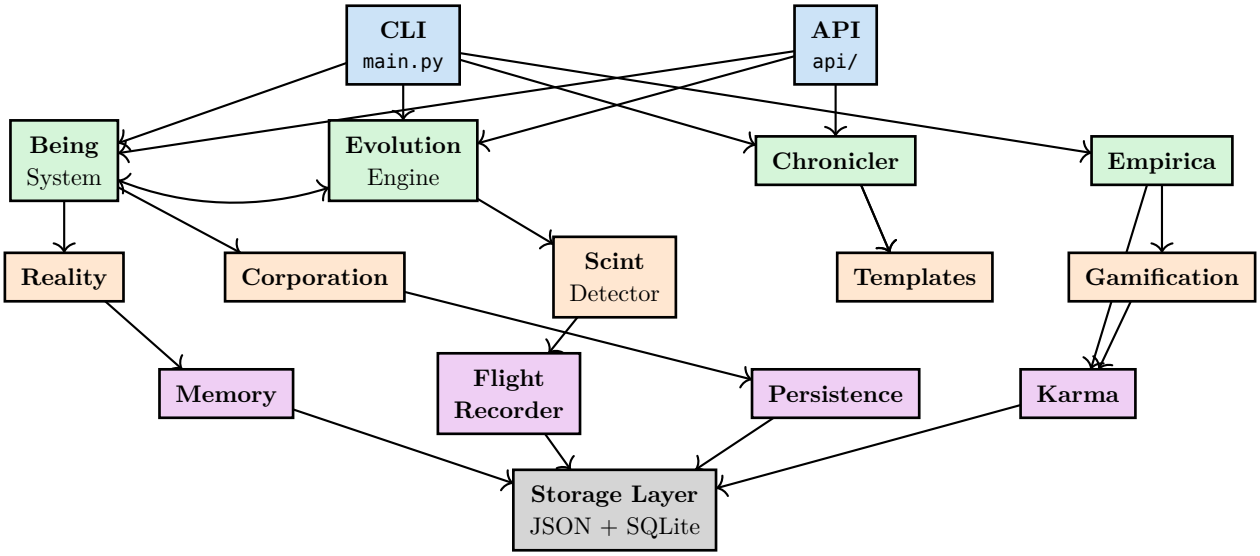
COMPONENT DIAGRAM

Module Relationships & Dependencies

Core Module Relationships



Module Dependency Matrix

	Being	Corp	Evol	Chron	Empirica
Being	—	uses	uses		
Corporation	refs	—		uses	
Evolution	evals		—	logs	tracks
Chronicler	reads	reads	reads	—	
Empirica			informs		—

Pantheon (Entity Gods)

Bureaucracy God
Corporate documentation, forms, procedures

GitHub God
Repository management, version control

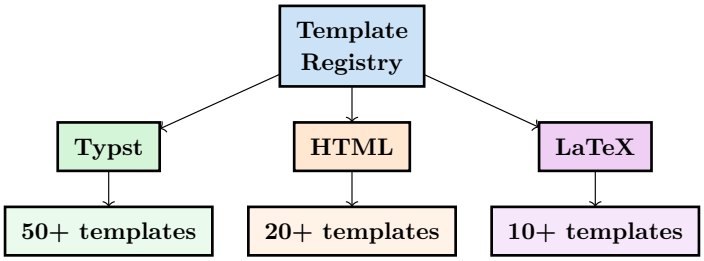
Paperwork God
Document generation, templates

Storyteller
Narrative generation, lore

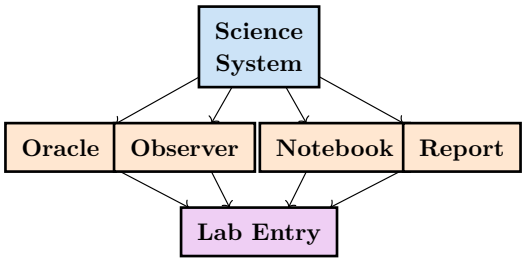
Judge
Evaluation, assessment

Librarian
Knowledge organization, retrieval

Template System

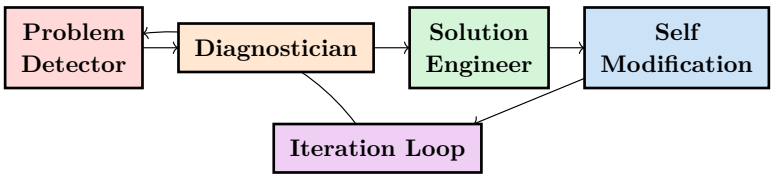


Science Module



Self-Engineering Module

The self-modification system for continuous improvement:



Full Module Map

Package	Modules	Purpose
`core/`	169	Business logic, domain models
`core/agent/`	6	Agent anatomy and state
`core/chronicler/`	6	Documentation generation
`core/corporations/`	12	Economic simulation
`core/dnd_scenario/`	11	RPG scenario generation
`core/dnd5e/`	6	D&D 5e mechanics
`core/hub/`	5	Central coordination
`core/science/`	12	Research framework
`core/self_engineering/`	9	Self-modification
`core/tavern_keeper/`	5	NPC management
`core/tracing/`	7	Observability
`evolution/`	28	Evolutionary algorithms
`pantheon/`	18	Entity gods
`templates/`	51	Document templates
`api/`	22	REST endpoints
`ui/`	13	Dashboards and visualizations

Interface Contracts

Key Interfaces

```python
# Every major component implements these patterns:

class Persistable(Protocol):
    def to_dict(self) -> dict: ...
    def from_dict(cls, data: dict) -> Self: ...
    def save(self, path: Path) -> None: ...
    def load(cls, path: Path) -> Self: ...

class Evolvable(Protocol):
    def fitness(self) -> float: ...
    def mutate(self) -> Self: ...
    def crossover(self, other: Self) -> Self: ...

class Observable(Protocol):
    def subscribe(self, observer: Observer) -> None: ...
    def notify(self, event: Event) -> None: ...
```

COMPONENT DIAGRAM | 443+ Modules, One Vision